Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Carcassonne – Python

Rapport préliminaire

Anthony BEAUVIEUX, Tony HUANG, Maxence SALIN, Clément SIMON, Loïc THIN

1 Présentation du jeu

1.1 Existant

Carcassonne est un jeu de stratégie et de placement de tuiles créé par Klaus-Jürgen Wrede en 2000, conçu pour 2 à 6 joueurs. Il s'agit d'un jeu stochastique à information parfaite, c'est-à-dire un jeu où le hasard intervient dans le tirage des tuiles mais où l'ensemble des éléments présents sur le plateau est visible de tous les joueurs.

Le jeu a connu un succès commercial important donnant lieu à de nombreuses extensions et variantes au fil des années. Il est devenu un classique des jeux de société modernes, alliant stratégie, gestion de ressources et placement de tuiles. Sur le plan académique et technique, Carcassonne représente un terrain d'étude privilégié pour l'intelligence artificielle (IA) et les algorithmes de décision, en raison de sa nature stochastique, de son système de scoring complexe et de la diversité des stratégies possibles. Plusieurs travaux académiques et implémentations logicielles ont étudié les problématiques liées à l'intelligence artificielle pour Carcassonne. L'article de Heyden et al.[6] propose une analyse détaillée de l'arbre de jeu et des techniques de recherche heuristique adaptées aux contraintes de ce jeu, tandis que d'autres études explorent des approches basées sur Monte Carlo ou Expectimax pour gérer l'incertitude liée à la pioche des tuiles.

1.2 Contexte



FIGURE 1 – Tuile de départ

La version considérée ici utilise les 72 tuiles du jeu de base. Parmi ces tuiles, se trouve la tuile de départ (figure 1) qui est placée au centre du plateau avant le début de la partie. Cette tuile est la même pour chaque partie et comporte une *City* en haut (couleur marron), une *Road* qui traverse la tuile de gauche à droite et un *Field* en bas. Chaque joueur dispose de 7 pions de sa couleur pour marquer des points au cours de la partie. Le but du jeu est que les joueurs construisent progressivement le plateau, à tour de rôle, en posant des tuiles de manière cohérente et en plaçant stratégiquement leurs pions pour accumuler des points. Le vainqueur est le joueur qui totalise le plus de points à la fin de la partie.

1.3 Règles

Les règles de **Carcassonne** utilisées sont celles de la version 2012[3].

1.3.1 Spécificités

Tuile

Chaque tuile possède quatre côtés ainsi qu'un centre. Chaque côté correspond à un type de terrain parmi *Road*, *City* ou *Field*. Le centre peut représenter une *City*, un *Field*, une *Banner* ou un *Monastery*.



FIGURE 2 – Exemple d'une tuile où des *Fields* possèdent un *Monastery*



FIGURE 3 – Exemple d'une tuile où une *City* possède une *Banner*

Pion

Lorsqu'un pion est placé sur une tuile, il prend un rôle en fonction du type de terrain occupé : *Knight* sur une *City*, *Farmer* sur un *Field*, *Thief* sur une *Road* et *Monk* sur un *Monastery*.

1.3.2 Déroulement d'un tour

Les joueurs jouent à tour de rôle selon un ordre prédéfini (généralement dans le sens horaire, en commençant par le joueur le plus jeune). Lors de son tour, un joueur effectue les actions suivantes dans l'ordre indiqué :

1. **Piocher une tuile :** Le joueur pioche une tuile dans la pioche actuellement active. Cette tuile est visible par l'ensemble des joueurs.
2. **Placer la tuile :** La tuile piochée doit être placée sur le plateau en respectant les règles suivantes :
   - La tuile doit être adjacente à au moins une autre tuile déjà posée.
   - Les côtés en contact doivent correspondre à des terrains de même type.
3. **Placer un pion (optionnel) :** Le joueur peut placer un pion sur un élément de terrain de la tuile qu'il vient de poser (*City*, *Road*, *Field*, *Monastery*).
4. **Restriction de placement des pions :** Si l'élément de terrain choisi est relié à un élément du même type déjà occupé par un pion, aucun nouveau pion ne peut y être placé. Les *Road* et *City* agissent comme des séparateurs entre les *Field*.
5. **Résolution des terrains complétés :** Si la pose de la tuile permet de compléter un ou plusieurs terrains, ceux-ci sont résolus immédiatement et les joueurs concernés marquent des points selon certaines règles. (voir partie 1.3.3)
6. **Retour des pions :** Tous les pions associés à des *City*, *Road* ou *Monastery* complétés durant le tour actuel sont retirés du plateau et rendus à leurs propriétaires. Les pions *Farmer* restent en jeu jusqu'à la fin de la partie.

1.3.3 Score

Les points sont attribués à deux moments :

- Pendant la partie : dès qu'un terrain (*City*, *Road*, *Monastery*) est complété (figure 4).
- À la fin de la partie : pour les terrains incomplets (*City*, *Road*, *Monastery*) ainsi que pour les *Field*.

Les joueurs ajoutent alors à leur score les points correspondant aux terrains selon les règles suivantes :

Road :
Une *Road* rapporte 1 point par tuile la composant au joueur possédant le plus de pions (*Thief*) sur cette *Road*. En cas d'égalité, tous les joueurs concernés marquent ces points.

City :
Une *City* complète rapporte 2 points par tuile qui la compose ainsi que 2 points par *Banner* présent, pour le joueur possédant le plus de pions (*Knight*). Si la *City* est incomplète à la fin de la partie, chaque tuile rapporte 1 point et chaque *Banner* rapporte également 1 point. En cas d'égalité, les points sont attribués à tous les joueurs concernés.

Monastery :
Un *Monastery* rapporte 1 point par tuile ayant un sommet ou côté en commun, y compris la tuile du *Monastery*. Le score maximal d'un *Monastery* est donc de 9 points.

Field :
Chaque ensemble de *Fields* connectés ou isolés est attribué au joueur possédant le plus de pions (*Farmer*). Il marque alors 3 points par *City* complète adjacente aux *Field*. En cas d'égalité, tous les joueurs concernés marquent ces points.

A l'issue du décompte final, le joueur ayant obtenu le plus grand nombre de points remporte la partie.

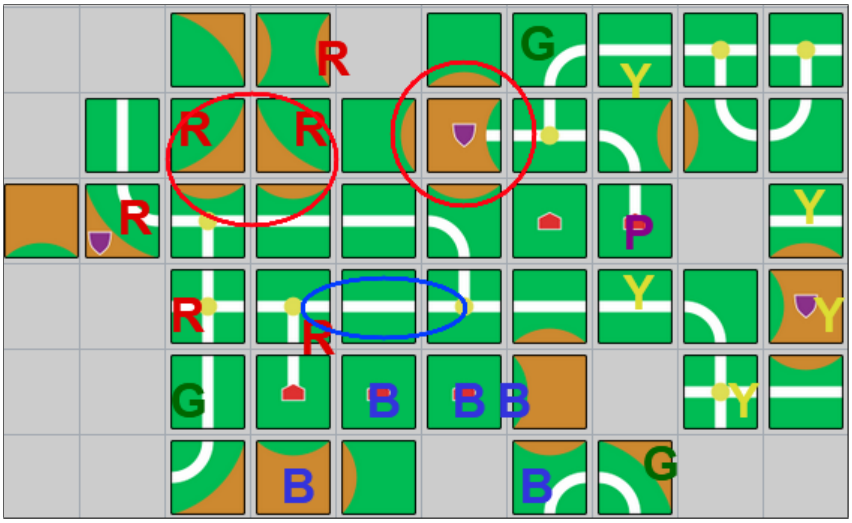


FIGURE 4 – Exemple de terrains complétés, avec deux *City* entourées en rouge et une *Road* entourée en bleu

1.4 Complexité

Bien que les règles de **Carcassonne** soient relativement simples, le jeu présente une richesse stratégique importante qui augmente à mesure que le plateau s'étend. Les choix offerts aux joueurs se multiplient progressivement en fonction de la configuration du plateau, des tuiles restantes et du placement des pions.

Contrairement à des jeux déterministes comme les échecs ou le shogi, **Carcassonne** introduit une composante stochastique via la pioche des tuiles, ce qui complexifie l'anticipation des coups futurs. Néanmoins, il demeure un jeu à information parfaite dans la mesure où l'ensemble des éléments placés sur le plateau est visible par tous les joueurs à tout instant.

La complexité combinatoire du jeu est particulièrement élevée pour plusieurs raisons. D'abord, chaque tuile peut être orientée de quatre manières et chaque position libre sur le plateau peut être un emplacement

légitime lorsque les côtés adjacents correspondent. Ensuite, la taille effective de l'espace d'états dépend de l'ordre de tirage des 72 tuiles, du placement des meeples et de la résolution des structures incomplètes. Sur un plateau en croissance, le nombre de successeurs possibles à un instant donné peut rapidement dépasser plusieurs dizaines, ce qui conduit à un arbre de jeu de taille astronomique. La taille de l'arbre de jeu est estimée à environ 10^{194}[6], ce qui rend toute exploration exhaustive irréaliste.

Mathématiquement, ce type de complexité se situe entre la simple permutation des tuiles et un vaste graphe de configurations : il faut combiner 72 choix de tuiles avec 4 orientations possibles et un grand nombre de positions valides, d'où un nombre de branches extrêmement élevé. D'où la nécessité d'un compromis entre profondeur de recherche, précision des heuristiques et temps de calcul.

Dans ce contexte, l'utilisation d'algorithmes de recherche heuristique et d'approximation apparaît comme une solution nécessaire pour proposer des décisions pertinentes dans un temps de calcul raisonnable.

2 Algorithmes et structures de données

Dans le cadre de ce projet, **Carcassonne** présente plusieurs caractéristiques qui complexifient l'application directe des algorithmes classiques de recherche dans les jeux. En particulier, la pioche de tuiles introduit une composante stochastique et le jeu peut accueillir jusqu'à cinq joueurs, ce qui invalide les hypothèses habituelles des formulations à deux joueurs à somme nulle.

Pour pallier ces contraintes, les décisions sont évaluées à partir de fonctions heuristiques estimant l'avantage relatif du joueur IA par rapport à ses adversaires à un instant donné. Cette modélisation permet l'utilisation d'algorithmes de recherche adaptés, tout en restant consciente de leurs limites théoriques dans ce contexte.

Les algorithmes suivants jouent un rôle central dans la réalisation des besoins du projet :

- **Minimax**[5], pour déterminer les coups optimaux en modélisant l'opposition entre les joueurs ;
- **Alpha-Beta**[7] qui réduit le nombre d'états explorés en élaguant les branches non pertinentes ;
- **Star2.5**, une extension d'Alpha-Beta aux jeux stochastiques via des nœuds de chance, retenu comme algorithme principal ;
- **Iterative Deepening**[9], permettant de borner le temps de calcul tout en améliorant progressivement la qualité des décisions ;
- des **fonctions d'évaluation heuristiques**, servant à estimer la qualité d'un état de jeu intermédiaire ;
- des approches basées sur *Monte Carlo Tree Search*[2], envisagées comme stratégies alternatives ;
- le mélange de *Fisher–Yates*[4], garantissant une pioche équitable.

La mise en œuvre de ces algorithmes repose sur des structures de données adaptées, détaillées dans la section suivante. Le plateau est modélisé par un graphe non orienté offrant une représentation naturelle des relations d'adjacence entre tuiles, complété par des composantes connexes pour la gestion des terrains et du scoring. La pioche et le retour des pions s'appuient sur des structures séquentielles simples telles que des listes et des piles.

2.1 Structures de données

La représentation des données du jeu repose principalement sur un graphe non orienté où chaque tuile constitue un sommet connecté à ses voisins cardinaux. Chaque sommet possède un identifiant unique correspondant à l'ordre de pose et des positions en coordonnées cartésiennes (x, y) à partir du centre de la tuile de départ.

Structure d'un sommet. Chaque sommet est représenté par la classe `Vertex` qui encapsule l'ensemble des informations associées à une tuile posée :

```
@dataclass
class Vertex:
    id: int                          # Identifiant unique de la tuile
    name: str                        # Code de la tuile (ex: "CRFR", "FSRS/m")
    orientation: int                 # Rotation (0-3)
    position: tuple[int, int]        # Coordonnées (x, y) sur le plateau
    neighbors: Dict[int, int]        # Arêtes vers les tuiles adjacentes
    matrix: list[list]               # Matrice 3x3 des terrains orientée
```

Chaque sommet est associé à une matrice de terrain 3×3 encodant le type de terrain (Cité `C`, Route `R`, Champ `F`, Monastère `M`, Stream `S`) selon sa rotation. La classe `Tile` fournit les quatre matrices de direction correspondant aux quatre rotations possibles :

```
class Tile:
 direction_matrix = [
 [["NW", "N", "NE"], ["W", "C", "E"], ["SW", "S", "SE"]], # Rotation 0 (0°)
 [["SW", "W", "NW"], ["S", "C", "N"], ["SE", "E", "NE"]], # Rotation 1 (90°)
 [["SE", "S", "SW"], ["E", "C", "W"], ["NE", "N", "NW"]], # Rotation 2 (180°)
 [["NE", "E", "SE"], ["N", "C", "S"], ["NW", "W", "SW"]], # Rotation 3 (270°)
 ]
```

En complément du graphe de tuiles, le système maintient une structure de composantes connexes par type de terrain :

```
lands_components = {
 'C': {'set_C1': [(tile_id_1, ['N', 'NE', 'E']), (tile_id_2, ['S'])], ...},
 'R': {'set_R1': [(tile_id_1, ['S', 'W']), ...], 'set_R2': [...], ...},
 'F': {'set_F1': [...], ...},
 }
```

Cette structure permet d'identifier rapidement les routes, cités et monastères complétés pour le scoring, de détecter les conflits de placement de meeples et de gérer la fusion de groupes lorsqu'une nouvelle tuile connecte des composantes existantes.

Mise à jour du graphe. Lors du placement d'une nouvelle tuile, la classe `GraphLogic` orchestre la mise à jour du graphe en quatre phases successives : connexion aux voisins cardinaux (N, E, S, W), connexion aux coins diagonaux (NE, SE, SW, NW), création des composantes isolées sans voisin existant, puis fusion des composantes connexes partageant un même terrain.

```
class GraphLogic:
 def add_lands_components(self, tile_id, graph, tiles_info):
 # Groupes de composantes selon leur connexité et leurs terrains
 group_is_linked = { ... }
 group_connection_count = { ... }
 group_land_type = { ... }
```

```
self._process_cardinal_neighbors(...) # Connexion aux voisins cardinaux
self._process_corner_neighbors(...) # Connexion aux coins
self._create_unlinked_groups(...) # Création des composantes isolées
self._merge_connected_groups(...) # Fusion des composantes connexes
```

Données bas niveau. En complément de `GraphLogic` qui applique les règles de connexité métier, la classe `GraphData` centralise toutes les opérations bas niveau de manipulation du graphe :

```
class GraphData:
    vertices: Dict[int, Vertex]                          # Sommets du graphe
    positions_to_vertex_id: Dict[tuple[int,int], int]   # Position -> id
    turn: int                                            # Compteur de pose
    lands_components: Dict[str, Dict[str, list]]         # Composantes connexes
    next_set_id: Dict[str, int]                          # Prochains ids disponibles
    meeples: List[Meeple]                                # Meeples sur le plateau
    tiles_dict: Dict[str, Tile]                          # Définitions des tuiles
```

Cette classe assure l'ajout et la suppression de tuiles orientées (`add/remove_vertex`), la gestion des arêtes entre tuiles (`set_neighbor`), la recherche de voisinage à une position donnée (`find_neighbor_at`), ainsi que la gestion des meeples (`add/remove/set_meeple`) et la réinitialisation complète du graphe (`reset_graph`)

2.2 Mélange des tuiles

La gestion de la pioche repose sur deux algorithmes de mélange, tous deux implémentés *in place* sur la liste représentant la pioche.

L'algorithme principal retenu est *Fisher-Yates*[4] qui garantit une permutation uniforme des tuiles lorsqu'il est associé à un générateur pseudo-aléatoire uniforme. Un algorithme naïf est également conservé à des fins de comparaison expérimentale afin d'illustrer les biais qu'un mauvais mélange peut introduire. Les propriétés statistiques de ces deux algorithmes sont analysées plus en détail en section 5.2.2.

```
import random
@staticmethod
def _shuffle_fisher_yates_in_place(
    items: list[str], rng: random.Random
) -> None:
    for i in range(len(items) - 1, 0, -1):
        j = rng.randrange(i + 1)
        items[i], items[j] = items[j], items[i]
@staticmethod
def _shuffle_naive_in_place(items: list[str], rng: random.Random) -> None:
    n = len(items)
    for i in range(n):
        j = rng.randrange(n)
        items[i], items[j] = items[j], items[i]
```

Dans la suite de cette section, nous nous concentrerons sur les algorithmes de prise de décision qui constituent le cœur du système d'intelligence artificielle implémenté dans le projet.

2.3 Algorithme Minimax

L'algorithme Minimax a pour but de déterminer les coups les plus avantageux pour un joueur en supposant que son adversaire agit de manière à minimiser son gain. L'algorithme parcourt récursivement une partie de l'arbre des états du jeu, limitée par une profondeur maximale fixée afin de contenir la complexité de recherche. Chaque feuille est évaluée via une fonction d'évaluation prédéfinie, et ce score remonte vers la racine. Chaque nœud sélectionne la valeur maximale ou minimale parmi ses enfants selon le joueur qu'il représente. L'algorithme prend en paramètres l'état courant du jeu, la profondeur restante à explorer et un booléen indiquant si c'est le tour du joueur max.

L'algorithme Minimax étant très similaire à $\alpha\beta$-pruning dans sa structure, nous invitons le lecteur à se référer à la section 2.4 pour une présentation détaillée accompagnée d'un exemple illustratif.

2.4 Algorithme Alpha-Beta

Alpha-Beta (ou $\alpha\beta$-pruning) est une variante de l'algorithme Minimax consistant à explorer partiellement l'arbre des états d'un jeu en élaguant les branches qui n'apportent aucun gain nouveau pour le joueur "max".[7] En stockant la meilleure valeur courante pour le joueur "max" (α) et celle pour le joueur "min" (β) lors du parcours, l'algorithme évite d'évaluer certains nœuds non pertinents (ainsi que leurs descendants potentiels) lorsque $\alpha \geq \beta$ ce qui peut réduire le temps d'exécution selon la structure de l'arbre.

Le principe est le suivant : dès qu'une branche s'avère trop défavorable pour l'un des joueurs, elle ne peut plus influencer la décision finale et peut être abandonnée. Le gain en temps est particulièrement important lorsque les successeurs sont ordonnés de manière à évaluer d'abord les coups les plus prometteurs.

```
fonction alpha_beta(etat, profondeur, alpha, beta, joueur_max):
    SI profondeur = 0 OU estFeuille(etat) ALORS
        RETOURNER evalueHeuristique(etat)
    FIN SI
    SI joueur_max ALORS
        valeur = -infini
        POUR TOUT successeur s de etat FAIRE
            valeur = max(valeur, alpha_beta(s, profondeur - 1, alpha, beta, FAUX))
            alpha = max(alpha, valeur)
            SI alpha >= beta ALORS
                BRISER
            FIN SI
        FIN POUR TOUT
    SINON
        valeur = +infini
        POUR TOUT successeur s de etat FAIRE
            valeur = min(valeur, alpha_beta(s, profondeur - 1, alpha, beta, VRAI))
            beta = min(beta, valeur)
            SI beta <= alpha ALORS
                BRISER
            FIN SI
        FIN POUR TOUT
    FIN SINON
    RETOURNER valeur
FIN FONCTION
```

À la profondeur maximale (Figure 5), les feuilles remontent la valeur 4 lors du tour du joueur max qui cherche à maximiser son gain. Le joueur min récupère cette valeur et la retient à 4 plutôt que 8 afin de minimiser le gain de son adversaire.
Le nœud max transmet alors 4 comme borne β au nœud min parent qui la propage à son tour à l'enfant max suivant. Cet enfant obtient la valeur 8 ; or $8 \geq 4$ ce qui signifie que le nœud min parent ne retiendra jamais cette valeur. L'exploration de cette branche est donc interrompue : la valeur 2 est élaguée.
Finalement, le joueur max retient la valeur 4 plutôt que 2, maximisant ainsi son gain minimal. La recherche se termine à la racine de l'arbre : l'algorithme a déterminé une succession de coups optimaux permettant d'atteindre un score de 4 à la profondeur demandée.

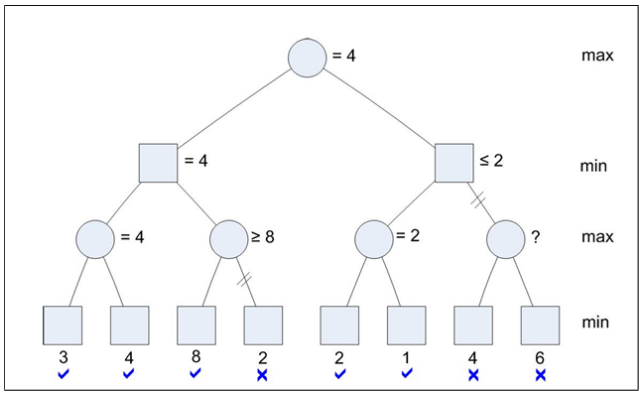


FIGURE 5 – Exemple d'un arbre de jeu exploré avec coupes par Alpha-Beta[6]

Cette version conserve la qualité de la décision de Minimax, tout en réduisant significativement le nombre de nœuds évalués lorsque le tri des coups est efficace.

2.5 Algorithme Star2.5

Dans Carcassonne, chaque joueur doit piocher une tuile au début de son tour. Cette action aléatoire remet en cause les hypothèses de base des algorithmes présentés précédemment. Nous avons donc étudié une variante prenant en compte les événements aléatoires : *Expectimax*. Cet algorithme fonctionne comme Alpha-Beta, mais introduit entre chaque étage de l'arbre des *nœuds de chance*, dont chaque fils représente un événement aléatoire possible depuis l'état courant — ici, le tirage d'une tuile particulière.

L'ajout de nœuds de chance à chaque niveau de l'arbre augmente cependant considérablement sa taille, rendant son parcours exhaustif impraticable dans le cadre d'un algorithme chronométré. Plusieurs techniques de pruning ont été développées pour pallier ce problème.

Star1. Star1 exploite les bornes α et β déjà présentes dans Alpha-Beta pour borner l'évaluation des nœuds de chance. Plutôt que de calculer exactement l'espérance de tous les enfants d'un nœud de chance, une borne inférieure et une borne supérieure de cette espérance au fur et à mesure de l'exploration sont maintenues. Si, à un moment donné, il devient certain que la valeur finale du nœud de chance ne pourra pas influencer la décision du parent — c'est-à-dire qu'elle est déjà en dehors de l'intervalle $[\alpha, \beta]$ — l'exploration des enfants restants est interrompue. Cela permet d'éviter d'explorer des tirages de tuiles peu pertinents.

Star2. Star2 améliore cette approche en resserrant davantage les bornes utilisées pour le pruning. Là où Star1 se limite aux bornes globales α et β, Star2 exploite également des bornes locales sur les valeurs possibles des feuilles, telles que les valeurs minimales et maximales de la fonction heuristique. Ces informations supplémentaires permettent de calculer plus précisément l'intervalle dans lequel se situe l'espérance finale d'un nœud de chance, déclenchant ainsi des coupures plus tôt dans l'exploration. Star2 se révèle donc plus efficace que Star1, en particulier lorsque le nombre d'événements aléatoires est élevé.

Star2.5. Star2.5 introduit une approche séquentielle : plutôt que de considérer les enfants d'un nœud de chance comme un ensemble non ordonné, ils sont parcourus dans un ordre spécifique, en priorisant par exemple les événements les plus probables ou les plus influents. À chaque étape, une estimation de l'espérance ainsi que des bornes optimistes et pessimistes pour les contributions restantes est mise à jour. En explorant en priorité les événements les plus significatifs, on obtient plus rapidement une estimation suffisamment précise

pour déclencher un pruning, maximisant ainsi les chances de coupure précoce.

```
fonction expectimax_star25(etat, profondeur, alpha, beta, joueur_max):
  SI profondeur = 0 OU estFeuille(etat) ALORS
    RETOURNER evalueHeuristique(joueur_max, etat)
  FIN SI
  SI estNoeudChance(etat) ALORS
    valeur = 0
    borne_min = 0
    borne_max = 0
    // Trier les événements (par probabilité décroissante par exemple)
    evenements = trierEvenements(etat)
    POUR TOUT (ev, proba) dans evenements FAIRE
      v = expectimax_star25(resultat(etat, ev), profondeur-1, alpha, beta, joueur_max)
      valeur = valeur + proba * v
      // Mise à jour des bornes restantes
      borne_min = valeur + sommeProbaRestante(evenements) * valeurMinPossible()
      borne_max = valeur + sommeProbaRestante(evenements) * valeurMaxPossible()
      // Pruning
      SI borne_max < alpha ALORS
        RETOURNER borne_max
      FIN SI
      SI borne_min > beta ALORS
        RETOURNER borne_min
      FIN SI
    FIN POUR TOUT
  RETOURNER valeur
  FIN SI
  // mettre algo alpha beta appelant expectimax_star25 à la place de alpha beta
```

Ce pseudo-algorithme illustre l'intégration des nœuds de chance avec une stratégie de pruning avancée, combinant les idées de Star2 et l'optimisation séquentielle propre à Star2.5.

Dans l'exemple de la figure 6, on considère un nœud de chance avec une fenêtre $\alpha\beta$ initiale de $[-10, 10]$ et deux événements possibles de probabilités 0.8 et 0.2. En suivant l'événement de probabilité 0.8, on obtient une valeur de -20 au nœud min. Une première estimation du nœud de chance est alors calculée en supposant le cas le plus favorable pour la partie restante :

$$0.8 \times (-20) + 0.2 \times 100 = 4$$

Cette valeur étant dans la fenêtre $[-10, 10]$, aucun élagage n'est encore possible.

En explorant ensuite l'événement de probabilité 0.2, le premier successeur retourne la valeur 10. Le nœud de chance met à jour sa borne inférieure :

$$0.8 \times (-20) + 0.2 \times 10 = -14$$

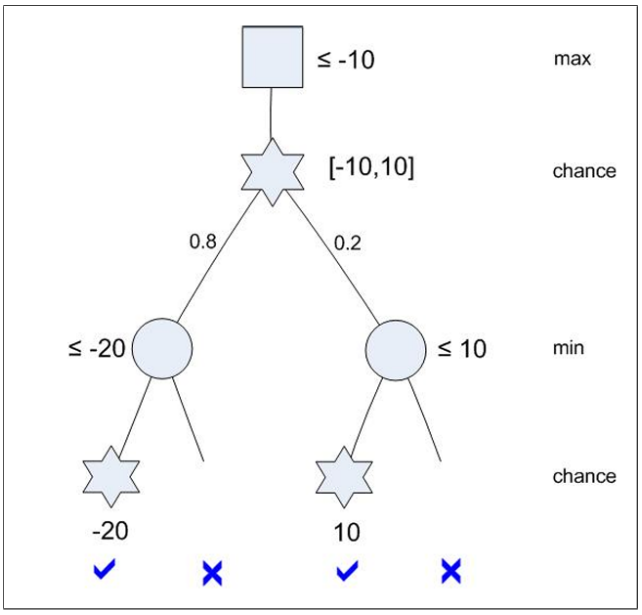


FIGURE 6 – Exemple d'arbre de jeu partiel avec Star2 menant à une coupe[6]

Cette estimation est désormais inférieure à -10, donc en dehors de la fenêtre $\alpha\beta$. La valeur finale de ce nœud de chance sera nécessairement trop faible pour être retenue par le joueur max : les branches restantes sont élaguées. Le nœud max parent rejette alors cette branche et poursuit la recherche ailleurs, évitant ainsi d'explorer inutilement une partie de l'arbre tout en conservant un résultat optimal.
Star2.5 est donc l'algorithme retenu en lieu et place du minimax avec $\alpha\beta$-pruning classique et sert de base à l'approfondissement itératif. Par ailleurs, Carcassonne se jouant de 2 à 5 joueurs, les algorithmes ont été adaptés pour accueillir plusieurs adversaires. Le joueur IA appelant l'algorithme est considéré comme le joueur max, tandis que tous les autres joueurs sont traités comme des joueurs min. Une branche de l'arbre de jeu se développe alors selon la séquence suivante : un nœud max, suivi d'un nœud chance, puis d'autant de nœuds min qu'il y a de joueurs adverses, avant de revenir à un nœud chance puis à un nœud max.

2.6 Technique Iterative Deepening

L'algorithme Minimax avec approfondissement itératif (Iterative Deepening) est une technique de parcours progressif des profondeurs sur l'arbre des états d'un jeu. Après un parcours avec une nouvelle profondeur explorée, le meilleur coup pour le joueur "max" est enregistré en remplaçant le précédent et sera alors renvoyé après épuisement du temps donné. Cette technique a pour avantage de déterminer une solution utile même sur une courte durée, tandis que l'exploration classique ne garantit pas que la profondeur fixée soit suffisante pour obtenir un résultat fiable après un temps bloquant.

En pratique, la recherche commence à faible profondeur puis augmente progressivement jusqu'à atteindre le temps limite :

```
fonction choix_coup(etat, temps_max):
    profondeur = 1
    meilleur_coup = null
    debut = temps()
    TANT_QUE temps() - debut < temps_max FAIRE
        meilleur_coup =  expectimax_star25(etat,
        profondeur, -infinite, +infinite, TRUE)
        profondeur += 1
    FIN TANT_QUE
    RETOURNER meilleur_coup
FIN FONCTION
```

Cette approche garantit qu'un coup plausible est toujours disponible à la fin du délai, même si la profondeur maximale n'a pas pu être atteinte.

2.7 Fonction d'évaluation heuristique

Étant donné l'impossibilité d'explorer l'arbre de jeu jusqu'aux états terminaux, l'algorithme Star2.5 repose sur une fonction d'évaluation heuristique permettant d'estimer la qualité d'un état de jeu intermédiaire. Cette fonction associe une valeur numérique à un état donné, représentant l'avantage relatif du joueur max par rapport à l'ensemble de ses adversaires.

L'évaluation repose sur plusieurs critères pondérés selon leur importance stratégique : le score actuel des joueurs, le potentiel de complétion des routes, cités et monastères en cours, le nombre de pions disponibles pour chaque joueur, la présence de pions bloqués sur des structures difficilement complétables et enfin le contrôle des champs adjacents à des cités complètes.

La fonction ne prétend pas fournir une estimation parfaite, mais guide efficacement la recherche vers des coups jugés prometteurs. Comme mentionné en section 5.2.1, certaines notions stratégiques avancées — comme le blocage adverse ou la gestion optimale des meeples — ne sont pas encore modélisées, ce qui constitue une limite connue de l'implémentation actuelle.

2.8 Lien avec le projet

Les différentes techniques présentées dans cette section sont combinées afin de rendre la recherche de coups pertinents possible malgré la complexité combinatoire élevée de Carcassonne. L'approche adoptée mêle Star2.5 — qui étend Alpha-Beta aux jeux stochastiques via des nœuds de chance — l'approfondissement itératif et une fonction d'évaluation heuristique multi-joueurs où le joueur IA est considéré comme joueur max et ses adversaires comme joueurs min.

La modélisation du plateau sous forme de graphes, associée à des structures de données adaptées comme les tables de hachage et les piles, assure une manipulation efficace de l'état du jeu et des configurations intermédiaires explorées lors de la recherche.

Bien que l'exploration exhaustive de l'arbre de jeu reste impossible, le système mis en place fournit des décisions de qualité en exploitant de manière pragmatique les ressources computationnelles disponibles. Cette combinaison d'algorithmes constitue le cœur de l'intelligence artificielle du projet et offre une base solide pour des extensions futures, notamment l'enrichissement des fonctions d'évaluation ou l'intégration de variantes du jeu.

3 Architecture

Dans cette section, nous présentons l'architecture logicielle retenue pour l'implémentation du jeu Carcassonne. Le projet adopte une architecture n-tiers[1] visant à séparer clairement les responsabilités afin d'améliorer la maintenabilité, la testabilité et l'évolutivité du code. Le principe fondamental de cette architecture est de diviser le système en couches distinctes, chacune ayant des responsabilités spécifiques et de limiter les dépendances au minimum grâce à des interfaces bien définies.

Dans un modèle n-tiers classique, chaque couche n'utilise que les services exposés par la couche située en dessous d'elle et ne dépend pas des détails d'implémentation de cette couche. Cela permet, par exemple, de remplacer l'interface utilisateur (CLI ou GUI) sans toucher à la logique métier, ou de simuler la persistance en tests unitaires en fournissant un faux service de stockage.

Ainsi, dans notre projet, nous avons choisi une architecture à trois tiers avec une séparation nette entre la présentation, la logique métier et les données. Cette architecture est pertinente dans notre cas car elle permet de faire évoluer le jeu sans mélanger les règles du jeu, l'affichage et la persistance des données. Elle est structurée autour de trois modules principaux : Présentation, Logique Métier et Données, organisés selon des dépendances unidirectionnelles.

3.1 Découpage en modules

Module Présentation

Le module Présentation est responsable des interactions avec l'utilisateur et de l'affichage de l'état du jeu. Il comprend notamment la gestion des commandes à travers le shell en CLI ou des actions à partir de la GUI, l'affichage du plateau, des tuiles, des meeples et des scores à travers CLI ou GUI.
Ce module ne contient aucune règle métier. Il délègue toute décision concernant la validité des actions ou l'évolution du jeu au module Logique Métier.

Responsabilités principales :

- Interprétation des actions utilisateur
- Appels aux interfaces de la logique métier
- Rafraîchissement de l'affichage

Module Logique Métier

Le module Logique Métier constitue le cœur du jeu. Il encapsule l'ensemble des règles du jeu et gère l'état interne de la partie. Il est notamment responsable de la mise à jour du plateau (via une liste d'adjacence), du calcul des scores, de la gestion des tours de jeu et des conditions de fin de partie. Il prend également en charge les actions des intelligences artificielles, à l'aide de fonctions d'évaluation, ainsi que la gestion de l'état du serveur et des parties en cours.
Ce module est totalement indépendant de l'interface graphique et de la persistance.

Responsabilités principales :

- Implémentation des règles du jeu
- Gestion de l'état de la partie
- Gestion des IA de jeu
- Gestion des actions liées au serveur

Module Données

Le module Données est dédié à la persistance et à la gestion des informations liées au jeu. Il s'occupe de la sauvegarde et du chargement des parties, de l'enregistrement de l'historique de jeu, ainsi que de la conservation des données relatives aux intelligences artificielles, notamment les structures de données et les meilleurs coups mémorisés dans le cadre de l'iterative deepening. Il gère également la persistance des parties en cours, de l'état du serveur et des informations associées aux joueurs.
Ce module n'implémente aucune règle métier et n'interagit jamais directement avec l'interface utilisateur.

Responsabilités principales :

- Sauvegarde des informations liées au jeu de près ou de loin (avec IA et serveurs) ;
- Possibilité de sauvegarder les parties en cours pour les recharger (dans des fichiers au format répondant aux fonctionnalités).

3.2 Dépendances entre modules

Les dépendances entre modules sont strictement unidirectionnelles (figure 7), conformément au principe d'architecture N-tiers.

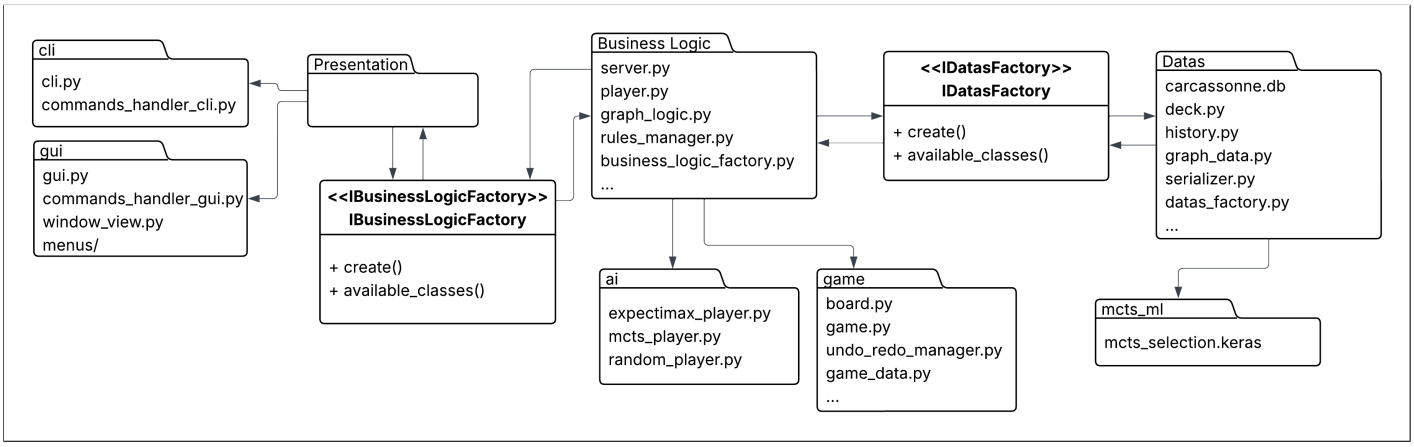


FIGURE 7 – Vue d'ensemble de l'architecture du projet

Ainsi, le module Présentation dépend uniquement du module Business Logic via l'interface `IBusinessLogicFactory` qui permet d'instancier les objets du module Business Logic et d'en appeler les méthodes sans que la Présentation n'accède directement aux données. De manière analogue, le module Business Logic dépend uniquement du module Données via `IDatasFactory` pour la persistance des informations. Aucune dépendance directe n'existe entre la Présentation et les Données.

Cette organisation garantit une séparation claire des responsabilités et limite le couplage entre les composants.

4 Explication approfondie

Notre projet de Carcassonne en Python a été bien différent de ce que nous avions initialement prévu notamment en raison de la complexité du jeu et des défis techniques rencontrés. Dans cette section, nous allons expliquer en détail les différentes étapes de notre développement, les choix que nous avons faits ainsi que les difficultés que nous avons rencontrées pour deux fonctionnalités principales.

4.1 Première fonctionnalité : Joueur artificiel

Parmi les intelligences artificielles implémentées, nous avons développé l'algorithme MCTS (Monte Carlo Tree Search). Les fichiers correspondants à son implémentation se trouvent dans le module Business Logic, plus précisément dans le fichier `mcts_player.py`.

Pour appliquer cet algorithme, nous avons représenté les nœuds de l'arbre à l'aide de la classe `MCTSNode`. Celle-ci contient l'état courant du jeu (`state`), son parent éventuel (`parent`), le coup ayant mené à ce nœud (`move`), le joueur associé à ce nœud (`player`), ainsi que le joueur MCTS à l'origine de son appel (`mcts_player`). Elle inclut également la liste de ses enfants (`children`), les coups encore non explorés depuis ce nœud (`untried_moves`), le nombre de visites (`visits`) et le score cumulé (`total_reward`). Des Getters sont définis pour chacun de ces attributs ainsi que deux méthodes dédiées à la mise à jour du nombre de visites et du score.

La classe `MCTSPlayer` centralise le déroulement complet de l'algorithme et construit progressivement l'arbre d'exploration via les `MCTSNode`. Elle inclut notamment l'état courant du jeu (`game`), les informations temporelles liées au tour de jeu de l'IA (à savoir le temps de départ, la durée maximale autorisée et le temps de fin : `start_time`, `max_time` et `end_time`) afin d'interrompre l'algorithme si la limite est atteinte. Elle

intègre également une fonction de sélection des enfants (`eval_fn`), un modèle de sélection basé sur le machine learning (`ml_model`), un booléen indiquant si ce modèle doit être utilisé comme fonction de sélection (`_ml_model_attempted`), ainsi qu'une structure de stockage permettant d'enregistrer les scores au fil des simulations et d'en permettre la restauration pendant l'exécution de l'algorithme (`_sim_state_stack`).

Le fonctionnement de l'algorithme repose sur la répétition de cinq étapes principales (Figure 8) :

- **Sélection :** à partir de la racine, on descend dans l'arbre en choisissant les enfants selon une fonction de sélection, jusqu'à atteindre un nœud à explorer.
- **Expansion :** un nouveau coup est sélectionné parmi ceux non encore explorés, ce qui crée un nouvel enfant.
- **Simulation :** une partie est simulée de manière aléatoire à partir de ce nouvel état.
- **Récompense :** le score final de la simulation est récupéré.
- **Rétropropagation :** les résultats sont propagés en remontant l'arbre jusqu'à la racine, en mettant à jour les statistiques des nœuds.

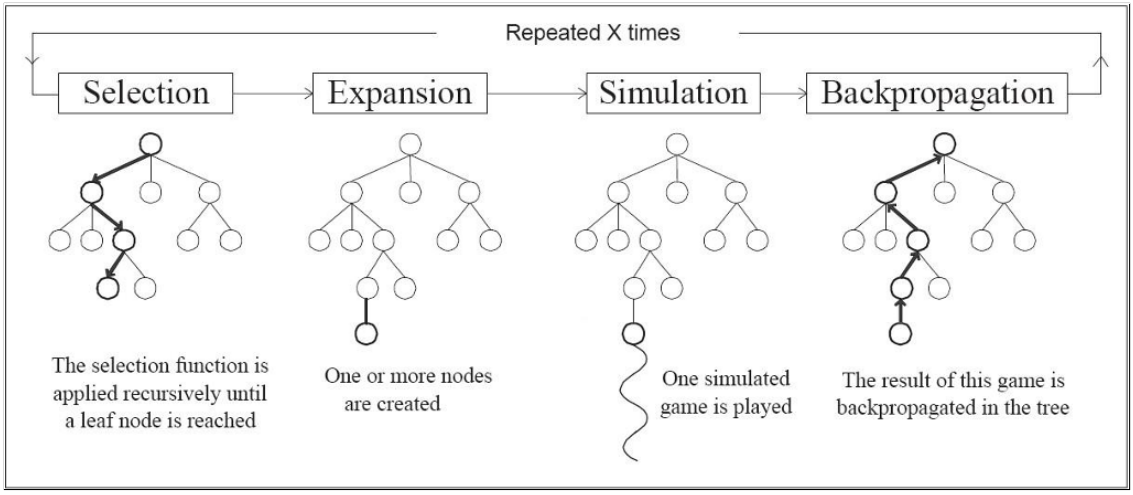


FIGURE 8 – Exemple d'application de l'algorithme MCTS[6]

À l'issue de l'algorithme, dont le fonctionnement est détaillé par le pseudo-code ci-après, le coup choisi correspond au nœud enfant de la racine présentant le meilleur score.

```
fonction mcts (etat, t0, budget, racine)
        TANT QUE tempsActuel() - t0 < budget FAIRE
    // 1) SELECTION
    noeud = racine
    TANT QUE noeud.estEntierementDeveloppe() ET NON noeud.estTerminal() FAIRE
        noeud = selection(noeud)
    FIN TANT QUE
    // 2) EXPANSION
    SI NON noeud.estTerminal() ALORS
        action = choisirActionNonEssayee(noeud)
        etat_enfant = appliquerAction(noeud.etat, action)
        enfant = ajouterEnfant(noeud, etat_enfant, action)
        noeud = enfant
    FIN SI
    // 3) SIMULATION (ROLLOUT)
```

```
    recompense = simulationAleatoire(noeud.etat)
    // 4) RETROPROPAGATION
    TANT QUE noeud != null FAIRE
        noeud.visites = noeud.visites + 1
        noeud.gain_total = noeud.gain_total + recompense
        noeud = noeud.parent
    FIN TANT QUE
  FIN TANT QUE
  // Choix final: meilleur enfant de la racine (souvent max visites)
  meilleur = argmax_enfant(racine, critere = visites)
  RETOURNER meilleur.action
```

Les fonctions `play_move_for_ai` et `undo_for_ai` permettent de simuler des coups sans altérer définitivement l'état réel du jeu. Lorsqu'un coup est joué via `play_move_for_ai`, l'état courant (scores, meeples, structures du graphe, etc.) est sauvegardé avant application puis stocké dans une pile. À l'inverse, `undo_for_ai` restaure précisément cet état en annulant le dernier coup joué. Ce mécanisme garantit que chaque simulation reste indépendante et n'impacte jamais la partie réelle.

La fonction `get_possible_moves` génère l'ensemble des coups possibles à partir de l'état courant et de la tuile en main. Elle consiste à tester toutes les positions, rotations et placements de meeples possibles, puis à vérifier leur validité à l'aide de la fonction `check_move` définie dans `rules_manager.py`.

La sélection des nœuds dans l'arbre peut se faire via l'algorithme UCT (Upper Confidence Bound applied to Trees)[8]. Cette méthode équilibre exploitation (favoriser les coups déjà performants) et exploration (tester de nouvelles possibilités) grâce à un paramètre de pondération. Elle prend en compte le nombre de visites et les scores des nœuds, tout en priorisant les nœuds encore jamais explorés afin d'assurer une couverture suffisante de l'espace des coups possibles.

Une alternative à la méthode UCT consiste à guider la phase de sélection de MCTS par un modèle de machine learning [10]. Pour chaque nœud enfant, un vecteur de caractéristiques est construit (reward moyen, nombre de visites, score, présence d'un meeple, etc.), identique à celui utilisé lors de l'entraînement. Le modèle prédit alors un score pour chaque coup et le coup ayant la meilleure évaluation est sélectionné. En cas d'indisponibilité du modèle, le système bascule automatiquement vers la méthode UCT, garantissant ainsi la robustesse de l'ensemble.

Entraînement et inférence. Les phases d'entraînement et d'inférence sont explicitement séparées. L'entraînement est réalisé hors ligne via le script `mcts_ml_training.py`, à partir de données issues de simulations de parties. Chaque exemple du dataset est constitué d'un couple *(features, target)*, où les features décrivent un état ou un nœud MCTS et la target correspond à une mesure de qualité (reward final ou estimation de victoire). Les données sont stockées au format JSON Lines, puis converties en tableaux numériques via NumPy. Le fichier d'exemple fourni constitue une validation technique et ne reflète pas encore un dataset issu de parties réelles.

Architecture du modèle. Le modèle utilisé est un perceptron multicouche (MLP) composé de deux couches cachées avec activation ReLU, suivi d'une couche de sortie linéaire adaptée à la régression. Ce choix permet de capturer des relations non linéaires tout en conservant un coût de calcul faible, compatible avec une utilisation en temps réel dans MCTS. L'entraînement minimise l'erreur quadratique moyenne (MSE) à l'aide de l'optimiseur Adam, sur un nombre d'epochs configurable. Contrairement à la phase de jeu, cet entraînement n'est pas contraint par `ai-time` et s'exécute jusqu'à convergence ou arrêt manuel.

Utilisation en partie. Une fois entraîné, le modèle est sauvegardé au format `.keras` puis rechargé dynamiquement pendant les parties via `ai_model.py`. Lors de l'exécution de MCTS, les mêmes features sont reconstruites à la volée pour chaque nœud exploré, garantissant la cohérence entre apprentissage et inférence. Le modèle est utilisé exclusivement en prédiction, sans mise à jour des poids.

Prérequis, sous-besoins, dépendances, intégration dans le projet, tests et validation. Les besoins F1, F4 et F15 assurent un shell interactif permettant de visualiser l'état du jeu. Les besoins F8, F9, F10, F11, F13 et F20 garantissent que l'algorithme MCTS dispose des éléments nécessaires à son fonctionnement : gestion de la pioche, nombre de joueurs, règles de placement, détection de fin de partie et possibilité d'annuler ou rejouer des coups pour faire évoluer l'arbre de recherche. Les besoins F26 et F27 permettent de stocker et de manipuler l'état du jeu via la structure *graph*, tandis que F31 borne le temps de réflexion de l'algorithme, garantissant qu'un résultat est toujours disponible à l'échéance.

Sur le plan fonctionnel, le joueur IA MCTS est encapsulé dans la classe `MCTSPlayer` qui permet de choisir entre les fonctions de sélection UCT et ML via les options `-ai-mode mcts` et `-ai-mcts-selection`. Le temps de réponse borné, assuré par F31, empêche le tour du joueur IA de dépasser la durée impartie.

Couverture de tests. La partie MCTS atteint une couverture de 90%. Les tests du fichier `mcts_player_test.py` valident le noyau algorithmique : structure des nœuds (état, parent, coup, statistiques), sélection de stratégie (UCT ou ML), condition terminale, génération des coups légaux, expansion, rollout, calcul de récompense et rétropropagation. Ils vérifient également la robustesse de la sélection ML face à plusieurs scénarios de repli : modèle absent, échec au chargement ou erreur en prédiction. Dans chacun de ces cas, l'algorithme bascule automatiquement sur UCT, sécurisant ainsi le mode ML dans les environnements où Keras ou le modèle ne sont pas disponibles.

La cohérence de l'état du jeu pendant les simulations est également vérifiée : jouer puis annuler un coup restaure correctement la main courante, le tour joueur, l'historique, les scores, les meeples et les composantes du graphe. Les scénarios de rollout couvrent les cas limites tels qu'une main injouable, une pioche nécessaire ou une pioche vide.

La chaîne d'entraînement du modèle ML est quant à elle couverte dans `mcts_ml_training_test.py` qui teste le parsing des datasets (JSON et JSONL), la validation des données invalides, ainsi que la construction, la compilation, l'entraînement et la sauvegarde du modèle.

Intégration dans l'architecture. `mcts_player.py` est intégré dans le package `ia` du module Business Logic, aux côtés de `mcts_ml_training.py` qui gère l'entraînement du modèle. Le modèle entraîné (`mcts_selection.` est stocké dans le module Données, conformément à l'architecture du projet.

Une limite subsiste cependant : pour valider pleinement cette fonctionnalité, des tests simulant des parties complètes avec un temps de réflexion suffisamment élevé seraient nécessaires. Ces tests étant trop coûteux en temps pour être intégrés à la suite pytest, la validation repose sur les tests unitaires existants qui assurent le bon fonctionnement de MCTS dans les cas couverts.

4.2 Seconde fonctionnalité : Jeu en réseau

Afin de permettre à plusieurs joueurs de s'affronter à distance, nous avons implémenté une fonctionnalité de jeu en réseau. Cette partie repose sur une architecture client-serveur : le serveur gère l'état global des parties et les interactions entre les clients, tandis que chaque client représente un joueur individuel. Cette fonctionnalité constitue un besoin essentiel pour offrir une expérience de jeu complète, fidèle à l'esprit du jeu de société dont ce projet est une adaptation numérique. Sa mise en place a nécessité que le jeu de base, les joueurs et le CLI soient préalablement fonctionnels, afin de pouvoir tester rapidement les échanges entre client et serveur.

L'implémentation est organisée autour de quatre composants principaux, répartis dans les modules du projet :

- `online_communications` (module Business Logic) : gère les réponses du serveur aux commandes des clients ;
- `games_repository` (module Datas) : représente la base de données et les opérations associées ;

- `server` (module Business Logic) : assure le fonctionnement global du serveur, incluant la gestion des clients, des parties et de l'état interne ;
- `client.py` (module Business Logic) : gère la communication avec le serveur et les interactions avec l'utilisateur.

Le schéma ci-dessous (figure 9) illustre les interactions entre ces composants, en mettant en évidence les flux de données et les canaux de communication.

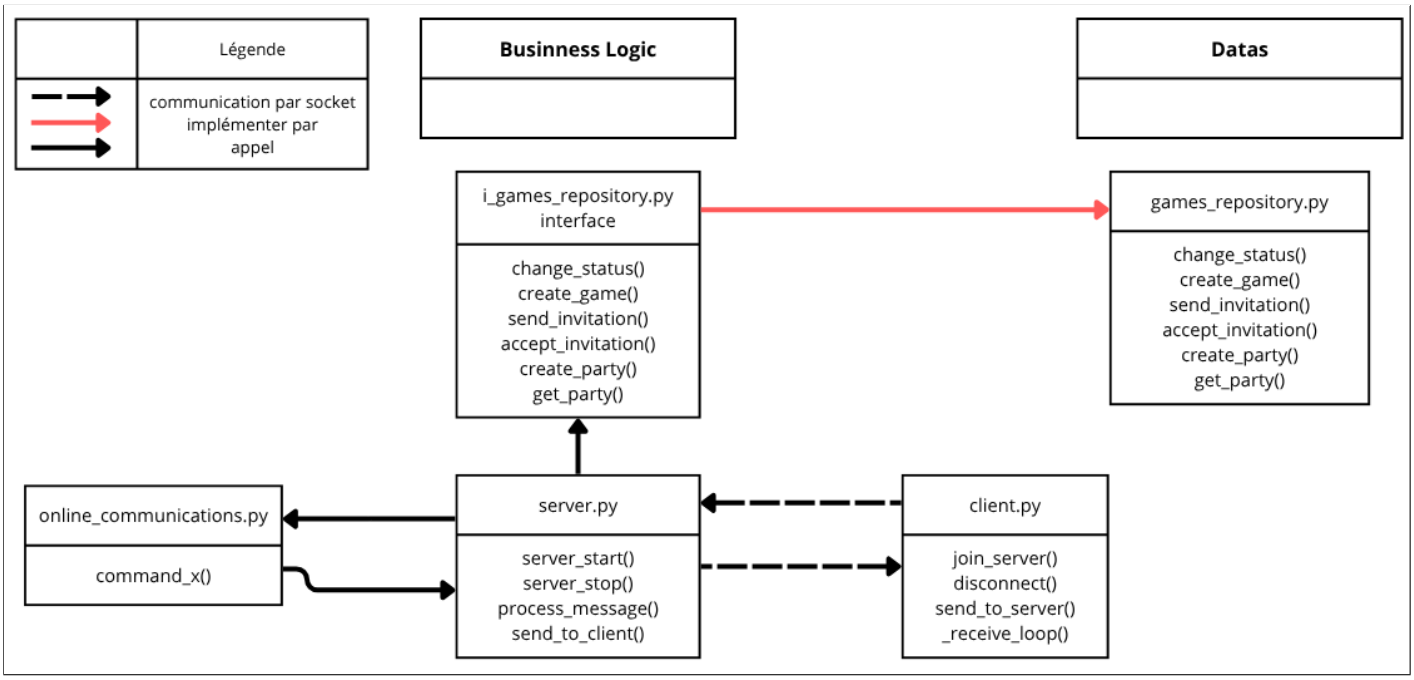


FIGURE 9 – Schéma représentatif de l'architecture du jeu en réseau

Les parties suivantes détaillent le fonctionnement de chaque composant et le rôle de leurs principales fonctions dans la gestion du jeu en réseau.

4.2.1 Serveur

La première fonction du serveur est `server_start`. Cette fonction démarre le serveur en TCP sur un port donné en argument. Si aucun port n'est donné, le port 12345 sera utilisé par défaut. Suite à cela, un thread pour le serveur est créé et démarré sur la fonction `server_running`.

La fonction `server_running` permet d'accepter des connexions clients tant que le serveur tourne. De plus, un timer est mis en place pour envoyer périodiquement les informations nécessaires aux clients afin qu'ils puissent se connecter. Cela est fait par UDP broadcast toutes les 10 secondes. Cette fonction prend aussi en charge les lancements et fins de parties ainsi que les time-out des invitations de chaque client. Finalement, chaque socket de client accepté est dirigé sur son propre thread vers la fonction `handle_client`.

Pour accomplir cette gestion, plusieurs fonctions internes sont utilisées :

La première est `check_all_invitations_timeout` qui renvoie la liste des clients ayant reçu une invitation dont le délai de réponse est dépassé. La fonction `launch_availables_games` permet de lancer toutes les parties pouvant l'être, c'est-à-dire les parties pour lesquelles les invitations ont toutes été résolues et le nombre de joueurs est correct.

Conjointement, la fonction `clean_finished_games` se charge de nettoyer les parties terminées, tandis que `check_clients_timeout` vérifie si les clients ayant effectué une déconnexion inopinée ont dépassé leur délai de time-out.

Chaque socket de client est ensuite redirigé dans son propre thread vers la fonction `handle_client` qui se charge de la communication entre le serveur et un client. Si le client souhaite quitter, un message est envoyé à tous les clients du serveur. La commande `quit` est la seule commande qui peut être effectuée sans être enregistré auprès du serveur. Si le client n'est pas encore enregistré, la fonction `handle_client_registering` se charge de l'enregistrer, et différents messages sont envoyés en fonction de la réussite ou de l'échec de l'enregistrement. Dans le cas d'un échec, deux messages principaux peuvent être envoyés : soit le nom d'utilisateur est déjà pris, soit le client est sous un time-out pour déconnexion inopinée.

Une fois le client enregistré auprès du serveur, celui-ci traite les messages du client et envoie les réponses appropriées via la fonction `process_message` qui utilise la classe `online_communications` afin de déterminer les réponses. La gestion de nombreuses erreurs est présente dans cette fonction et quoi qu'il arrive, les structures de données du serveur seront mises à jour.

Les dernières fonctions notables du serveur sont les suivantes : `send_to_group` et `send_to_client` qui permettent d'envoyer des messages respectivement à un groupe de joueurs ou à un joueur. On trouve également des accesseurs pour les différentes structures de données du serveur, ainsi que la fonction `server_stop`, appelée aussi bien lors d'un arrêt normal que lors d'un arrêt brutal du serveur. Cette fonction nettoie toutes les ressources proprement.

4.2.2 Client

Le fichier `client.py` sert principalement à envoyer et recevoir des informations depuis le serveur auquel il est connecté.

La première fonction utilisée est `create_default_online_player`. Elle se charge de créer un compte pour un serveur donné. Par défaut, il s'agit du serveur local. Toutefois, si un problème d'authentification survient sur un serveur externe, cette fonction permet de créer un compte spécifique pour ce serveur, en générant un nouveau nom d'utilisateur tout en conservant le même mot de passe.

La deuxième fonction rencontrée est `list_available_servers`. Elle écoute sur le port 12346 pour détecter les transmissions UDP broadcast des serveurs et constitue à partir de celles-ci une liste de serveurs disponibles. Tout serveur n'ayant pas envoyé de message depuis un certain temps est retiré de cette liste par la fonction `check_servers_last_time_sent` qui vérifie la date de la dernière diffusion de chaque serveur.

Ensuite, la fonction `join_server` permet de rejoindre un serveur figurant dans la liste des serveurs disponibles. Si aucun argument n'est fourni à cette fonction, elle récupérera les informations nécessaires auprès du serveur personnel, à condition que celui-ci figure dans la liste des serveurs disponibles ; sinon, aucune connexion ne sera établie. Lorsqu'une connexion est effectuée, un thread est créé côté client pour gérer la communication avec le serveur.

La fonction interne `_receive_loop` s'occupe de lire les messages tant que la connexion est active. Si elle reçoit un message du serveur indiquant un arrêt (*shutdown*), la boucle se termine. Dans le cas contraire, les messages sont transmis à la fonction `handle_server_message`. Cette fonction sert à transmettre les messages du serveur au GUI si celui-ci est disponible. Elle se charge également d'automatiser certains comportements du client. Par exemple, lorsqu'un client tente de se connecter pour la première fois et que le serveur renvoie un message indiquant que son nom d'utilisateur est déjà pris, la fonction offre au client la possibilité de modifier son nom d'utilisateur. En CLI, cette fonction affiche les messages du serveur sous une forme plus lisible.

Une autre fonction importante est `send_to_server`. Assez simple, elle transmet le message fourni en paramètre au serveur, selon le format suivant :

```
username: message
```

La fonction `connected_to_a_server` est une fonction servant la logique interne du client. Enfin, la fonction `disconnect` prend en charge les déconnexions brutes et normales et se charge de nettoyer toutes

les ressources de la session qui ne sont pas des ressources de la base de données. Il existe également la fonction `register_message_visitor` qui utilise l'idée du design pattern visitor afin de pouvoir capturer les messages envoyés par le serveur pour les transmettre au GUI.

4.2.3 Online_communications

La classe `OnlineCommunication` se charge de retourner les réponses appropriées aux commandes et d'effectuer les opérations demandées si elles peuvent l'être. Ses fonctions peuvent être séparées en deux catégories : les commandes spécifiques au serveur et les commandes spécifiques au jeu.

La commande `quit` demande à la base de données d'effectuer les modifications nécessaires pour déconnecter le client et renvoie ensuite le message correspondant. La commande `server status` transmet les informations pertinentes concernant le serveur. La commande `new ...`, quant à elle, envoie des invitations aux joueurs disponibles figurant dans la liste d'identifiants fournie. Si aucun joueur n'est disponible, une réponse appropriée est renvoyée.

La plupart des commandes dépendent de l'état du serveur et/ou de la base de données. Toutes les commandes liées au jeu ne produiront aucun effet et renverront un message adapté si le joueur n'est pas actuellement dans une partie. Enfin, la commande `ping` mérite une mention spéciale : elle se contente de renvoyer `pong`.

4.2.4 Base de données

La base de données constitue un élément central du serveur. Presque toutes les opérations importantes y passent, que ce soit pour récupérer des informations, en créer de nouvelles, les modifier, ou plus rarement les supprimer. La classe `games_repository` est conçue de manière à ce que le serveur et la logique métier n'aient pas à connaître la structure interne de la base de données.

Toutes les opérations sur la base de données reposent sur deux types de transactions : les transactions *query* qui sont des transactions de sélection de données et les transactions *non query* qui sont des transactions de modification des n-uplets. Ces opérations sont protégées contre toute injection de code SQL par des utilisateurs malveillants.

Pour répondre aux besoins de Carcassonne, plusieurs tables ont été créées. La première, et l'une des plus importantes, est la table des joueurs. Chaque joueur y possède un identifiant unique généré automatiquement par la base de données. La table contient également le nom d'utilisateur principal et le mot de passe. Le statut du joueur est inclus pour gérer correctement les événements et peut prendre l'une des quatre valeurs suivantes : `idle`, `away`, `waitgame` ou `ingame`, représentant les différents états possibles du joueur. Le numéro de groupe du joueur est également présent et est initialement à `null`. Enfin, deux booléens sont utilisés : l'un indique si le joueur est connecté et l'autre s'il est un client courtois, c'est-à-dire un client qui quitte le serveur correctement.

Pour représenter les groupes de joueurs dans chaque partie, la table `groups` contient l'identifiant auto-généré du groupe, le nombre de joueurs dans celui-ci et l'identifiant de l'hôte. La table `scoreboard` est plus simple. Elle stocke pour chaque joueur son identifiant, son nombre de victoires et de défaites. La table `current_games` conserve les parties en cours avec l'identifiant auto-généré de la partie, son nom et le groupe associé.

L'une des tables les plus importantes pour la gestion des parties est la table `pending_invitations`. Chaque invitation y possède un identifiant auto-généré, l'identifiant du groupe concerné, l'identifiant du joueur invité et le statut de l'invitation qui peut être soit `waiting` soit `accepted`. La date d'envoi de l'invitation est également enregistrée afin de gérer le temps maximum de réponse.

Pour gérer les conflits de noms d'utilisateurs entre différents serveurs, la table `usernames` a été créée. Elle contient l'adresse du serveur et le nom d'utilisateur associé. Si un serveur n'est pas référencé dans cette table, le nom d'utilisateur principal est utilisé par défaut, sous réserve qu'il ne soit pas déjà pris sur ce serveur et que la phase d'enregistrement ait déjà eu lieu. Cette table est la seule à contenir des données propres au propriétaire

de l'ordinateur. Enfin, la table `timeouts` enregistre les identifiants des joueurs qui se sont déconnectés de manière inattendue ainsi que la date à laquelle cette déconnexion a été constatée par le serveur.

Le jeu en réseau est basé sur les besoins F38 à F40, tous les besoins ont étaient effectués en CLI, mais pas en GUI. En effet, la partie GUI a été développée en fin de projet et nous n'avons pas eu le temps de l'intégrer complétement avec les fonctionnalités de jeu en réseau. Cependant, l'implémentation en GUI du besoin F38 est complète.

5 Revue critique du projet

Ce projet nous a confrontés à plusieurs défis majeurs notamment la complexité des règles de Carcassonne et la mise en œuvre des intelligences artificielles. Cette section propose une analyse critique de notre travail, organisée en cinq parties : le respect du cahier des charges, les performances obtenues, les limites et bugs rencontrés, les pistes d'amélioration envisageables et enfin l'usage des outils d'intelligence artificielle au cours du développement.

5.1 Cahier des charges

Dans cette section, nous allons discuter de la manière dont nous avons respecté le cahier des charges du projet en analysant chaque partie et en expliquant ce qui a été réalisé et ce qui n'a pas été fait.

5.1.1 Généralités

Cette première partie du projet a été globalement respectée. Des tests ont été mis en place tout au long du développement afin de vérifier le bon fonctionnement du code et de s'assurer que les différentes fonctionnalités étaient correctement implémentées. Avant le rendu, nous avons également vérifié la compatibilité des bibliothèques utilisées avec les exigences du cahier des charges.

Grâce à cette approche, le projet atteint une couverture globale d'environ 90% ce qui constitue un résultat conforme aux exigences du cahier des charges.

5.1.2 Besoins généraux

La principale difficulté de cette partie a résidé dans la mise en place du fichier de configuration du besoin F2 en lien avec l'ensemble des fonctionnalités supplémentaires ajoutées au fil du projet. Un problème subsiste cependant. Le cahier des charges stipule que le fichier de configuration doit être lu au démarrage et ses options utilisées par défaut. Or, une fois ce fichier présent, il n'est plus possible de revenir aux options par défaut du programme sans modifier manuellement le fichier. Une commande dédiée à la réinitialisation des options par défaut aurait été nécessaire pour pallier ce manque.

5.1.3 Options au lancement

Les besoins portaient principalement sur des options de lancement relativement faciles à implémenter. Il a néanmoins fallu intégrer différents modes de jeu comme le mode *Blitz* ou *Contest* (F6 et F7) ainsi que l'extension *The River* (F10). Ces éléments ont été réalisés en fin de projet une fois les fonctionnalités de base implémentées ce qui a facilité leur intégration.

Une difficulté particulière a cependant été rencontrée lors de l'implémentation de *The River*, concernant la règle de virage. En effet, cette règle impose des contraintes sur l'orientation des tuiles rivière afin d'éviter que la rivière ne forme une boucle ce qui s'est révélé non trivial à vérifier lors du placement des tuiles.

Nous pouvons également mentionner les algorithmes de mélange de tuiles et leurs tests statistiques, détaillés en section 5.2.2.

5.1.4 Gestion du jeu

La gestion des règles de base du jeu a constitué la première grande étape du projet, mais aussi l'une des premières difficultés majeures. Les règles de Carcassonne sont en effet relativement complexes à implémenter : connectivité des tuiles, conditions de placement des pions, calcul du score. Chacun de ces aspects a nécessité une grande rigueur afin de garantir une implémentation correcte.

Le placement des tuiles et des pions a été relativement simple à traiter, notamment grâce à une bonne modélisation du plateau de jeu et des tuiles via le module *graph*. En revanche, la gestion du score a représenté un défi majeur. Elle a nécessité la mise en place de composantes connexes, en particulier pour les champs pouvant s'étendre entre routes et cités, dont la délimitation s'est révélée particulièrement délicate à formaliser. Cette partie a été source de nombreux bugs et a significativement ralenti l'avancement du projet.

5.1.5 Command Line Interface (CLI)

Le CLI a été relativement simple à implémenter, notamment grâce à la bibliothèque *readline* de Python qui facilite la saisie des commandes et a permis de répondre aisément à deux besoins principaux : l'édition de la commande en cours et la gestion de l'historique des commandes tapées. Il a toutefois été nécessaire de développer des fonctions spécifiques pour l'auto-complétion des commandes et de leurs arguments afin de satisfaire le besoin F19.

Une amélioration notable resterait la mise en place d'un design pattern dédié à la gestion des commandes (besoin F15). Une telle approche aurait permis de mieux structurer le système dès le départ et d'en faciliter l'évolution, notamment lors de l'implémentation des commandes dans le GUI, où l'absence de cette structure s'est fait ressentir.

5.1.6 Format de sauvegarde

Cette partie a été développée progressivement au fil de l'avancement du projet. Nous avons d'abord mis en place une sauvegarde basique de l'état du jeu avant d'y ajouter des fonctionnalités complémentaires, telles que la gestion du format de l'historique et la conformité avec le fichier de configuration. La représentation textuelle du plateau de jeu a, quant à elle, n'a pas revelé de problèmes ; la structure du plateau se prêtant naturellement à une expression sous forme textuelle.

5.1.7 Structures de données

La mise en place d'un graphe pour représenter le plateau de jeu a constitué la première étape du projet. Cette structure nous a permis de gérer efficacement les connectivités entre les tuiles, notamment pour le calcul du score des champs via des composantes connexes.

Cependant, certains choix effectués lors de cette implémentation précoce ont été source de difficultés par la suite. En particulier, les tuiles du plateau sont modélisées comme de simples sommets (`Vertex`) du graphe, sans classe `Tile` dédiée. Cette décision, anodine au départ, s'est révélée de plus en plus contraignante à mesure que le projet évoluait. L'absence de distinction claire entre une tuile et un sommet de graphe a alourdi la gestion du code notamment lors de l'implémentation du GUI où manipuler directement des tuiles en tant qu'objets aurait considérablement simplifié le développement.

5.1.8 Graphical User Interface (GUI)

L'implémentation du GUI a été, en parallèle de l'intelligence artificielle, la partie la plus longue du projet. Le principal défi a résidé dans la dépendance du GUI vis-à-vis des autres fonctionnalités du projet. Il a en effet fallu attendre que les différentes composantes soient suffisamment stables pour pouvoir les intégrer dans l'interface graphique. Le GUI s'est également révélé plus complexe à développer que le CLI, nécessitant la mise en place de nombreux éléments graphiques tels que des menus, des boutons, ainsi que l'affichage des tuiles et des meeples.

Une difficulté technique notable a été la gestion du placement des tuiles sur le plateau. Le GUI repose sur un système de coordonnées (x, y) pour représenter les positions, tandis que le module de logique métier utilise une référence par tuile et orientation. Il a donc fallu établir une correspondance entre ces deux représentations. Pour cela, nous avons mis en place une fonction permettant de retrouver la tuile voisine d'une position (x, y) donnée sur le plateau, utilisée ensuite comme entrée pour le *play_move* du module de logique métier.

Malgré ces avancées, certaines améliorations restent envisageables, notamment concernant l'adaptabilité du plateau à la taille de la fenêtre et la gestion exhaustive des erreurs utilisateur, pour rendre le GUI pleinement cohérent avec l'ensemble des besoins du projet.

5.1.9 Joueur artificiel

La partie concernant les joueurs artificiels a été la plus complexe du projet. En effet, comme abordé dans la section 2.3, Carcassonne est un jeu à la fois stochastique et à information parfaite ce qui rend l'implémentation de Minimax inutilisable dans sa forme classique. Il a fallu ainsi implémenter une version adaptée de Minimax avec des noeuds chance pour prendre en compte de la pioche des tuiles. Cette approche correspond à l'algorithme Expectimax, une extension de Minimax pour les jeux stochastiques. Cependant, l'implémentation de cet algorithme s'est avérée très complexe et a été source de nombreux bugs, notamment en ce qui concerne la gestion des noeuds chance et la mise en place de fonctions d'évaluation heuristiques adaptées à ce jeu.

5.1.10 Jeu en réseau

La partie la plus chronophage a été la gestion du serveur côté GUI. Plus précisément, il a fallu permettre au GUI d'afficher les messages envoyés par le serveur. Pour cela, nous avons mis en place le design pattern Visitor afin que le GUI puisse traiter et afficher ces messages sans établir de lien direct entre les deux composants.

La partie la plus complexe a, quant à elle, été la gestion des différentes commandes envoyées par le serveur. En effet, la base de données ayant évolué tout au long du projet, le comportement associé à chaque commande a dû être régulièrement revu. De plus, la base de données se devant de refléter fidèlement l'état réel du serveur, toute modification doit se répercuter de manière cohérente entre les deux, ce qui complexifie significativement la gestion des commandes.

Par ailleurs, la gestion des interruptions abruptes du serveur ou du client a été une source de nombreux bugs, nécessitant la mise en place de fonctions de nettoyage afin d'éviter les fuites de ressources et la corruption de la base de données.

Cependant, les besoins F39 et F40 n'ont été que partiellement réalisés côté GUI : si la réception et l'affichage des messages serveur fonctionnent correctement, le lancement d'une nouvelle partie via la commande `new` depuis le GUI reste non opérationnel. Cette fonctionnalité est en revanche pleinement prise en charge dans le CLI.

5.2 Performance

Cette section a pour objectif d'évaluer empiriquement les performances des différentes composantes du projet, en particulier les intelligences artificielles et certains choix algorithmiques. Nous analysons les résultats obtenus à travers plusieurs expérimentations afin d'identifier les comportements observés, leurs limites ainsi que les pistes d'amélioration possibles.

5.2.1 IA

IA	Wins (incl. ties)	Win rate	Avg score	Avg place
Iterative	9	60.0%	22.0	1.63
Random	3	20.0%	15.1	3.43
MCTS_ML	2	13.3%	14.3	3.20
Minimax	2	13.3%	14.3	3.33
MCTS_UCT	1	6.7%	14.7	3.40

TABLE 1 – Comparaison des performances des différentes IA sur 15 parties mixtes, chaque partie incluant une IA de chaque type.

La table 1 présente les performances des différentes intelligences artificielles sur 15 parties mixtes. Afin de garantir l'équité expérimentale, chaque IA a occupé chaque position de départ qui est considérée comme plus favorable à trois reprises sur les 15 parties, avec une alternance systématique des positions. L'IA Iterative domine nettement avec un taux de victoire de 60% et une position moyenne de 1.63, tandis que les autres IA, notamment les variantes MCTS, obtiennent des performances significativement plus faibles dans ce contexte compétitif.

Ce résultat est d'autant plus surprenant que des expérimentations préalables montrent que les IA basées sur MCTS atteignent entre 60% et 80% de taux de victoire lorsqu'elles sont évaluées contre des IA aléatoires. Ces performances ont été mesurées sous différentes contraintes de temps de calcul :

- 1 seconde : environ 25 itérations par tour (UCT) contre 15 (ML)
- 5 secondes : environ 112 itérations (UCT) contre 68 (ML)

Ce contraste met en évidence un point important : une IA performante en environnement contrôlé ne garantit pas de bonnes performances en contexte multi-agents compétitif. Plusieurs hypothèses peuvent expliquer ce phénomène. D'une part, les simulations MCTS reposent sur des modèles adverses simplifiés, souvent aléatoires, ce qui les rend moins robustes face à des stratégies variées. D'autre part, le coût computationnel élevé des simulations qui impliquent de rejouer toute la logique du jeu, limite le nombre d'itérations réalisables et donc la qualité des décisions prises. Enfin, les variantes MCTS implémentées (UCT et ML) peuvent ne pas être optimalement adaptées à ce type de jeu ou à ce format multi-joueurs.

5.2.2 Algorithme de mélange des tuiles

Un point plus technique concerne les algorithmes de mélange des tuiles dont la validation a nécessité deux niveaux de tests. Le premier repose sur des tests fonctionnels déterministes : pour un paquet initial donné, un même algorithme et une même graine doivent produire un résultat identique. Nous vérifions également que le mélange conserve exactement le multiensemble des tuiles, sans perte, duplication ni altération. Dans le cas du paquet « rivière », une contrainte supplémentaire impose que la tuile spéciale finale reste en dernière position. Ces tests garantissent la reproductibilité et l'intégrité des structures de données mais ne permettent pas à eux seuls d'évaluer la qualité statistique du mélange.

Pour analyser l'uniformité des algorithmes, nous avons étudié un cas simplifié composé de trois tuiles distinctes. Ce choix est justifié par le fait que le paquet réel contient des doublons, ce qui rend l'analyse des permutations plus difficile, tandis qu'un paquet de trois éléments permet une étude exhaustive et exacte. Avec Fisher-Yates, l'algorithme effectue successivement un choix parmi 3 possibilités puis un choix parmi 2 possibilités, soit $3 \times 2 = 6$ trajectoires, exactement le nombre de permutations possibles. L'énumération complète montre que chacune des 6 permutations est obtenue une seule fois : la distribution est donc uniforme. À l'inverse, l'algorithme naïf effectue 3 échanges avec à chaque fois 3 choix possibles, soit $3^3 = 27$ trajectoires. L'énumération exhaustive donne alors des effectifs $(5, 5, 5, 4, 4, 4)$ pour les 6 permutations, au lieu d'une

répartition uniforme. Cela montre que plusieurs suites d'échanges conduisent plus souvent à certaines permutations qu'à d'autres. Ces résultats confirment les propriétés théoriques des deux méthodes : FisherYates est uniforme lorsque les indices sont tirés uniformément, alors que l'algorithme naïf est structurellement biaisé. Nous avons donc retenu FisherYates comme algorithme par défaut, tandis que l'algorithme naïf est conservé à des fins de comparaison expérimentale.

5.3 Limites et bugs

Malgré nos efforts pour implémenter correctement les différentes fonctionnalités du projet, nous avons rencontré de nombreux bugs et limitations au cours du développement.

5.3.1 Limites

- Le taux de couverture des tests atteint environ 80%. Toutefois, ce chiffre ne prend pas en compte les tests liés à l'interface graphique. En effet, ces derniers sont particulièrement difficiles à automatiser (avec une couverture moyenne d'environ 25%). Il est donc pertinent d'envisager d'autres approches pour évaluer la qualité du GUI, comme des tests manuels ou des retours utilisateurs.
- Les fonctions d'évaluation heuristiques (F33) utilisées par les IA ne modélisent pas certaines stratégies avancées, comme le blocage adverse ou la gestion optimale des meeples restants. Cela limite leur efficacité en contexte compétitif multi-joueurs, comme l'illustrent les résultats présentés en section 5.2.1.
- Un certain nombre de constantes du projet ne sont pas centralisées et sont dispersées à travers les différents modules ce qui nuit à la maintenabilité du code. En effet, toute modification d'une valeur nécessite de retrouver et mettre à jour chaque occurrence manuellement, augmentant ainsi le risque d'incohérences et d'erreurs difficiles à détecter.
- Le module `graph` respecte globalement la structure attendue en liste d'adjacence avec gestion des composantes connexes (F26). Toutefois, il n'est pas totalement indépendant et intègre encore des éléments de logique métier (joueur, meeple, tuile), ce qui nuit à la séparation des responsabilités.
- Le serveur ne semble fonctionner correctement que dans l'environnement du Cremi. En effet, les tests effectués en réseau local n'ont pas été concluants, suggérant que le serveur repose sur des configurations réseau spécifiques à cet environnement. Cela constitue une limitation importante quant à la portabilité du mode réseau du projet.
- Pour certains besoins du cahier des charges, nous avons fait le choix d'une implémentation basée sur notre propre compréhension des spécifications, en adoptant la solution qui nous semblait la plus cohérente avec l'ensemble du projet. Ces choix peuvent ainsi ne pas correspondre exactement aux attentes initiales. C'est notamment le cas du besoin F39 concernant le mode daemon du serveur, que nous avons implémenté selon notre interprétation, après des échanges avec les chargés de TD. De même, le besoin F10 relatif à l'extension *The River* a fait l'objet de discussions avec les chargés de TD, dont les retours ont laissé entrevoir un écart entre leurs attentes et notre implémentation, suggérant que notre interprétation pourrait être à revoir.

5.3.2 Bugs

- Le besoin F12, relatif à la gestion des messages d'erreur, est implémenté dans le GUI, bien que certains cas restent non couverts, notamment lors du chargement d'une partie invalide.
- Les besoins F39 et F40 concernant les fonctionnalités avancées du serveur sont pleinement fonctionnels dans le CLI, mais partiellement implémentés dans le GUI. En particulier, la commande new permettant de lancer une nouvelle partie depuis le GUI n'est pas opérationnelle. Le message est bien envoyé au serveur mais la réponse n'est pas correctement traitée et affichée côté GUI. Cela s'explique par la complexité

du pattern Visitor mis en place qui n'a pas pu être étendu à temps à l'ensemble des types de messages du serveur.

5.4 Perspectives d'amélioration

Une fois le projet complété, nous avons identifié plusieurs pistes d'amélioration, que ce soit au niveau de l'architecture du code, de l'implémentation de certaines fonctionnalités ou de l'extensibilité du projet.

Architecture et qualité du code

L'amélioration la plus évidente concerne l'organisation générale du code. Un manque de prévoyance dans l'architecture initiale a conduit à des choix d'implémentation qui ont rendu certaines parties du projet difficiles à maintenir et à faire évoluer. Cela s'est illustré notamment lors de la restructuration des fichiers *game.py* et *gui.py* qui dépassaient chacun les 1000 lignes et concentraient trop de responsabilités distinctes. Nous avons tenté de remédier à ce problème au fil du développement, mais avons parfois dû faire des compromis pour avancer dans les délais.

Une meilleure réflexion en amont sur l'architecture du projet aurait permis d'éviter ces écueils. Par exemple, la mise en place d'un design pattern dédié à la gestion des commandes aurait structuré davantage le système et facilité l'implémentation des commandes dans le GUI. De même, une centralisation des constantes dès le début du projet aurait amélioré la cohérence et la maintenabilité de l'ensemble du code.

Fonctionnalités et extensibilité

Sur le plan fonctionnel, plusieurs aspects pourraient être améliorés. La gestion des cas limites, comme celui d'une tuile impossible à placer, mériterait d'être traitée explicitement afin de garantir un comportement défini dans toutes les situations. Cette même rigueur pourrait être appliquée à d'autres composantes du projet : la robustesse du serveur face à des situations réseau inattendues, l'expérience utilisateur du GUI notamment via un plateau adaptatif et une meilleure gestion des erreurs, ou encore les performances de nos IA grâce à des fonctions d'évaluation plus élaborées modélisant des stratégies avancées comme le blocage adverse ou la gestion optimale des meeples.

Nous avons ainsi réussi à implémenter un projet fonctionnel répondant à une grande partie des besoins du cahier des charges. Les améliorations restantes relèvent principalement de la qualité du code, de la robustesse de certaines fonctionnalités et de l'expérience utilisateur et ne remettent pas en cause le bon fonctionnement général du projet.

5.5 Utilisation de l'IA

Tout au long du projet, nous avons eu recours à des outils d'intelligence artificielle pour diverses tâches, que ce soit pour la mise en place d'un code de base, l'amélioration de la qualité du code ou l'affinement de certaines fonctionnalités. Cela nous a permis d'avancer plus rapidement et efficacement, tout en développant un regard critique sur les forces et les limites de chaque outil.

Nous avons largement utilisé *GitHub Copilot Chat* dans Visual Studio Code, en mode *chat* et *inline*. Cet outil s'est révélé particulièrement efficace pour les tâches répétitives et bien délimitées, comme la génération de fonctions utilitaires ou la complétion de blocs de code dont la structure était déjà claire. En revanche, ses suggestions pouvaient s'avérer longues à apparaître et souvent inadaptées lorsque le contexte fourni était insuffisant ou que la tâche demandée dépassait la simple complétion locale.

Pour des problématiques plus complexes, nous avons eu recours à *ChatGPT* (version gratuite, modèle GPT-4o mini) et à *Claude Sonnet 4.6* via claude.ai, notamment pour la mise en place du design pattern Visitor afin de permettre au GUI d'afficher les réponses du serveur. Ces modèles se sont révélés utiles pour structurer une approche face à des problèmes architecturaux ou pour clarifier des concepts. Toutefois, nous avons constaté

leurs limites lorsque le contexte du projet devenait trop volumineux. Les suggestions proposées étaient alors déconnectées des choix d'implémentation existants, nécessitaient un travail d'adaptation important, voire introduisaient de nouveaux bugs.

En fin de projet, nous avons également expérimenté le *vibe coding*, une pratique consistant à décrire un comportement souhaité en langage naturel et à laisser l'IA générer le code correspondant. Cette approche a été utilisée principalement pour peaufiner certaines fonctionnalités et corriger des bugs mineurs. Elle s'est avérée efficace pour des tâches bien circonscrites, mais a montré ses limites dès lors que les modifications impliquaient plusieurs composantes interdépendantes, l'IA peinant alors à respecter les contraintes de l'architecture existante.

Enfin, l'IA a également contribué à l'amélioration de ce rapport, en nous aidant à structurer nos idées, à formuler nos analyses de manière plus claire et concise et à corriger les fautes d'orthographe et de grammaire.

Références

[1] Nawar S Alseelawi, Enas K Adnan, Hussein T Hazim, Haider Alrikabi, and Khalid Nasser. Design and implementation of an e-learning platform using n-tier architecture. *International Journal of Interactive Mobile Technologies (iJIM)*, 2020.

[2] Fred Valdez Ameneyro, Edgar Galván, and Ángel Fernando Kuri Morales. Playing carcassonne with monte carlo tree search. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 2343–2350. IEEE, 2020.

[3] Christopher Obe. Carcassonne complete annotated rules. `https://www.dover.nh.gov/Assets/government/city-operations/library/borrow/library-of-things/carcassonne-rules.pdf`, 2014.

[4] Ronald A. Fisher and Frank Yates. Statistical tables for biological, agricultural and medical research. *Oliver and Boyd*, 1938.

[5] Thomas Hauk, Michael Buro, and Jonathan Schaeffer. Rediscovering*-minimax search. In *International Conference on Computers and Games*, pages 35–50. Springer, 2004.

[6] Cathleen Heyden. Implementing a computer player for carcassonne. *Master's thesis, Department of Knowledge Engineering, Maastricht University*, 2009.

[7] Donald E. Knuth and Ronald W. Moore. Analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4) :293–326, 1975.

[8] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo tree search. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.

[9] Richard E. Korf. Depth-first iterative-deepening : An optimal admissible tree search. *Artificial Intelligence*, 27(1) :97–109, 1985.

[10] Darpan Pandey, Kamal Niwaria, and Bharti Chourasia. Machine learning algorithms : a review. *Mach. Learn*, 6(2) :916–922, 2019.

6 Annexe : Liste des besoins

Besoin	Description du besoin	Statut
F1	Lancement du programme	Fait
F2	Fichier de configuration	Fait
F3	Internationalisation (i18n)	Fait
F4	Interface en ligne de commande	Fait
F5	Interface graphique	Fait
F6	Mode blitz	Fait
F7	Mode contest	Fait
F8	Graine aléatoire	Fait
F9	Mélange des tuiles	Fait
F10	Options spécifiques	Fait
F11	Gestion des règles	Fait
F12	Gestion des erreurs utilisateur	Partiel
F13	Gestion du undo/redo	Fait
F14	Gestion des conseils	Fait
F15	Shell interactif	Fait
F16	Proposer de sauvegarder avant quitter	Fait
F17	Édition de la commande en cours	Fait
F18	Historique des commandes tapées	Fait
F19	Complétion des commandes	Partiel
F20	Notation des coups	Fait
F21	Sauvegarde et restauration d'une partie	Fait
F22	Commentaires dans la sauvegarde	Fait
F23	Format des paramètres du jeu	Fait
F24	Format du plateau de jeu	Fait
F25	Format de l'historique	Fait
F26	Module *graph*	Fait
F27	Algorithmes de manipulation des graphes	Fait
F28	Menus de l'application	Fait
F29	Raccourcis clavier	Fait
F30	Drag'n'drop	Fait
F31	Temps de réflexion borné	Fait
F32	Minimax avec $\alpha\beta$-pruning	Fait
F33	Fonctions d'évaluation pour Minimax	Fait
F34	Approfondissement itératif de Minimax	Fait
F35	Profondeur de recherche de Minimax	Fait
F36	Monte Carlo Tree Search (MCTS)	Fait
F37	Fonction de sélection par Machine Learning	Fait
F38	Découverte de serveurs et connexion client- serveur basique	Fait
F39	Serveur multi-clients et gestion de parties	Partiel
F40	Système d'invitations et gestion avancée des joueurs	Partiel